

Reverse Engineering Stockfish 19 Evaluation Semantics and a Stability-Oriented Eval-Bar Design for Unchessed

Manus AI
Unchessed UCI Engine Research September 2026

Abstract—This report reverse engineers the publicly available Stockfish 19 evaluation-to-display pipeline and translates the findings into an implementation specification for a homemade evaluation bar in Unchessed. The analysis is pinned to the official `sf_19` tag, commit `edb0d9db6731067ec50ce619ff372b463bc4dd5d`, rather than moving development code. Stockfish 19 is shown to combine an NNUE network’s PSQT and positional outputs, a complexity-dependent optimism adjustment, material scaling, a rule-50 dampener, tablebase-safe clamping, side-to-move normalization, and a material-dependent logistic WDL model. The report also completes the attached Maia false-positive prompt by requiring a three-move clean fingerprint in the accelerated detector and records 31 passing focused regression tests. The resulting design deliberately separates a deterministic static score, a calibrated WDL bar, and search-confidence metadata; it does not claim exact Stockfish equivalence without the matching SFNNv16 network and source revision.

I. SCOPE, CONSTRAINTS, AND REPRODUCIBILITY

The work was performed only on the Git branch `manus/research-facilities`. The repository’s `main` branch was inspected but not edited. The attached prompt reported a resilient Maia false-confirmation gate that reduced false confirmations from 60% to 40% at a 20-game-per-arm scale, while identifying an open question: require a longer clean fingerprint or add a new signal. This report chooses the former because it is local, interpretable, cheap, and testable without changing the legacy detector.

All external source inspection was pinned to primary project sources. The source files used for exact equation extraction were downloaded from the official Stockfish 19 tag and stored with SHA-256 hashes in `docs/source-evidence`. The companion implementation evidence is the Unchessed Rust source and its focused unit-test output.

II. STOCKFISH 19 EVALUATION PROCEDURE

A. NNUE output and post-processing

The SF19 evaluator calls the network and receives two internal components, `psqt` and `positional`. It first forms

$$n = p + q, \quad (1)$$

TABLE I
PINNED SOURCE ARTIFACTS

Artifact	SHA-256 prefix / provenance
Stockfish 19 tag	<code>edb0d9db6731067ec50ce61...</code>
<code>evaluate.cpp</code>	<code>68a1675ef7ae9430...</code>
<code>score.cpp</code>	<code>c5877628d70984fe...</code>
<code>uci.cpp</code>	<code>a31fa10719b1d9ec...</code>
<code>network.cpp</code>	<code>95cafe51c0d05842...</code>

where p is the PSQT component and q is the positional component. It then defines a disagreement or complexity statistic

$$c = |p - q|. \quad (2)$$

The source adjusts optimism by $c/476$ and shrinks the raw NNUE value by $c/18236$. With $M = 534 N_P + N_{\text{nonpawn}}$, the material-scaled value is

$$v_0 = n + \frac{nM + 7675o}{91000}, \quad (3)$$

where o is the search optimism term. The rule-50 correction is linear:

$$v_1 = v_0 \left(1 - \frac{r_{50}}{199}\right). \quad (4)$$

Finally, the value is clamped inside the tablebase mate range. These equations are taken directly from `sf_19/src/evaluate.cpp`, lines 50–68 [1].

This is a critical distinction for an eval bar: the displayed value is not a direct readout of a single neural-network neuron, nor is a classical pawn value guaranteed. A faithful bar must either call the same evaluator or explicitly label itself as a proxy.

B. NNUE architecture and incremental state

The official NNUE documentation describes a sparse, king-relative feature representation and an accumulator that adds or subtracts first-layer weight columns when features change [2]. HalfKP-style features are indexed by king square, piece square, piece type, and piece color. Two perspectives are maintained, and the side-to-move perspective can be placed first in the combined input. King moves are special because the king-relative feature set changes broadly and requires an accumulator refresh. Quantized integer arithmetic makes incremental updates stable and fast, but exact output still depends on the

SFNNv16 feature layout, quantization constants, and network weights.

C. Perspective and centipawn conversion

Stockfish’s internal static value is from the side-to-move perspective. In the trace path, SF19 negates the value when Black is to move before converting it to a White-centric displayed score. The UCI conversion uses a material-dependent parameter a :

$$\text{cp} = \text{round} \left(100 \frac{v}{a(M)} \right). \quad (5)$$

The material count for this calibration is $P + 3N + 3B + 5R + 9Q$, clamped to $[17, 78]$ and normalized by 58. The cubic polynomials are

$$a(m) = (((-142.7205m + 372.3518)m - 340.7107)m) + 415.2349. \quad (6)$$

$$b(m) = (((5.9383m + 15.6127)m - 30.5782)m) + 69.6387. \quad (7)$$

These coefficients and the conversion are from `sf_19/src/uci.cpp`, lines 537–563 and 585–594 [3].

D. WDL model

The WDL probabilities are computed as

$$W(v, M) = \frac{1000}{1 + \exp((a(M) - v)/b(M))}, \quad (8)$$

$$L(v, M) = W(-v, M), \quad (9)$$

$$D(v, M) = 1000 - W(v, M) - L(v, M). \quad (10)$$

The official WDL model documentation explains that the displayed centipawn scale is calibrated to self-play outcome statistics rather than literal pawn value, and that material affects the calibration [4]. The UCI output is therefore a per-mille tuple W, D, L , not a universal probability independent of time control, engine population, or material.

III. COMPARATIVE MODEL ANALYSIS

Leela Chess Zero is materially different. Its network supplies both a value estimate and a policy distribution, and PUCT/MCTS backs up values through a search tree [5]. Its root value is therefore a search statistic, not a direct static score equivalent to Stockfish’s NNUE output. The comparison suggests that Unchessed should not mix Lc0-style visit statistics into a Stockfish-style bar. Instead, the bar should expose a static evaluation and optionally display search confidence separately.

IV. ATTACHED-PROMPT COMPLETION

The accelerated resilient gate previously required two consecutive low-loss observations and a bounded volatility fingerprint. The completion changes the requirement to

$$\text{low_loss_streak} \geq 3, \quad (11)$$

while preserving the existing rating, resilient score, evidence, good-mass, volatility, and catastrophic-error conditions. The change is intentionally narrow: it does not alter

TABLE II
MODEL IMPLICATIONS FOR AN EVAL BAR

Model	Primary output	Safe design lesson
Stockfish 19	NNUE static value, post-scaled and WDL calibrated	Reproduce perspective, scaling, material dependence, and mate precedence.
Lc0	Policy plus value used by PUCT/MCTS	Do not treat a tree-backed value as a static centipawn score.
Classical HCE	Hand-tuned material and positional terms	Useful fallback and diagnostics, but not a substitute for SFNNv16 calibration.

the legacy path, does not allow clock evidence to promote on its own, and remains default-off behind accelerated detection.

The focused Rust suite was run with the current stable toolchain. It executed 31 opponent-model tests, including the new `accelerated_resilient_requires_three_move_clean_streak` regression, with 31 passed and 0 failed. The broader package command also began the FIDE perft integration harness but did not finish within the available test window; this is recorded as an infrastructure/runtime limitation rather than reported as a full-suite pass. No Maia UCI runtime was available in the sandbox for a fresh 20-game reproduction, so the attached report’s 40-game field result is treated as prior evidence, while the new three-streak behavior is supported by deterministic unit tests.

V. IMPLEMENTATION SPECIFICATION FOR UNCHESSED

The experimental bar should have four separable layers. First, obtain a deterministic static value from the engine’s current evaluation path, with explicit White-centric normalization. Second, apply the SF19-inspired post-processing only when the corresponding inputs are available; otherwise expose a proxy flag. Third, calculate material-aware WDL using the cubic logistic model, preserving mate as a discrete state. Fourth, smooth only the presentation layer, not the underlying score, using a bounded exponential moving average and a maximum per-update movement. Search depth, node count, MultiPV agreement, and principal-variation churn should be presented as confidence metadata rather than silently changing the score.

A practical bar percentage is the expected score

$$S = \frac{W + 0.5D}{1000}, \quad (12)$$

with a visual midpoint at $S = 0.5$. The UI should show both the White-centric score and WDL tuple so users can distinguish “+0.50” from a claim of a 75% win chance. When the position is a forced mate, the bar must switch to mate mode rather than pass the mate sentinel through the logistic function.

VI. THREATS TO VALIDITY

Exact replication is impossible without the official SFNNv16 network file, exact build path, and matching source revision. WDL is calibrated to a specified self-play population and time control; it is not a human win probability. The attached Maia sample is stochastic and small, and the 31 passing tests verify policy invariants rather than field false-positive rates. Lc0 is included as a conceptual comparison, not as a second implementation target. These limitations are design inputs: the experimental bar will report provenance and confidence instead of implying unsupported certainty.

VII. CONCLUSION

Stockfish 19’s evaluation bar is the endpoint of a pipeline: NNUE components, complexity correction, material/optimism scaling, rule-50 damping, safe clamping, perspective normalization, and material-dependent WDL calibration. A robust homemade bar can reproduce the public mathematics and protocol semantics while remaining honest about the missing network-dependent portion. The completed attachment gate adds a stricter three-move consistency fingerprint and passes its focused regression suite. The next implementation step is therefore a branch-local, provenance-aware eval-bar module, not a claim of replacing Stockfish’s trained evaluator.

REFERENCES

- [1] Stockfish Team, “Stockfish 19,” Stockfish, Sep. 5, 2026. [Online]. Available: <https://stockfishchess.org/blog/2026/stockfish-19/>
- [2] official-stockfish, “NNUE,” Stockfish Documentation. [Online]. Available: <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>
- [3] official-stockfish, “Stockfish source at sf_19,” GitHub, commit edb0d9d. [Online]. Available: https://github.com/official-stockfish/Stockfish/tree/sf_19/src
- [4] official-stockfish, “WDL model,” GitHub. [Online]. Available: https://github.com/official-stockfish/WDL_model
- [5] official-stockfish, “UCI Protocol and Stockfish Commands,” Stockfish Documentation. [Online]. Available: <https://official-stockfish.github.io/docs/stockfish-wiki/UCI-Protocol-and-Stockfish-Commands.html>
- [6] Leela Chess Zero, “Technical Explanation of Leela Chess Zero,” Lc0 Development Wiki. [Online]. Available: <https://lczero.org/dev/wiki/technical-explanation-of-leela-chess-zero/>